

Introduction to Optimization Project 3

Jun Moon

junmoon@hanyang.ac.kr

(with Yuna Oh and J. S. Lee)

주의사항

- DQN구현 시 Hyperparameter들에 대한 원리 및 설정값에 대한 설명을 해주셔야 합니다.
- DQN Hyperparameter 들 변화에 대한 결과 변화 설명을 해주셔야 합니다.

Machine Learning (Deep Learning) 개발 환경

- Python
- PyTorch or Tensor Flow
- Machine Learning 개발환경 세팅 관련 링크
 - <https://theorydb.github.io/dev/2020/02/14/dev-dl-setting-local-python/>

PyTorch & Tensor Flow

- 인공지능을 개발하기위한 Python기반 개발 라이브러리
- **Pytorch**
 - Facebook에서 개발한 딥러닝 개발 라이브러리
 - 직관적이라서 익히기 쉬움
 - 학습된 모델 및 코드를 타 플랫폼(모바일, 웹 등)에 옮기기 어려움
- **Tensorflow**
 - Google에서 개발한 딥러닝 라이브러리
 - 학습된 모델 및 코드를 타 플랫폼(모바일, 웹 등)에 옮기기 용이함
 - 처음 익히기엔 덜 직관적

Join us at PyTorch Conference in San Francisco, October 22-23. CFP open now! [Learn more.](#)



Learn ▾

Ecosystem ▾

Edge ▾

Docs ▾

Blog & News ▾

About ▾

Become a Member



GET STARTED

Choose Your Path: Install PyTorch Locally or
Launch Instantly on Supported Cloud Platforms

Get started >

BLOG

Stay up-to-date on the latest news and technical topics from the PyTorch Foundation.

[Read more](#)

PYTORCH 2.6

Featuring Python 3.13 support for torch.compile, several AOTInductor enhancements, FP16 support on X86 CPUs, and more.

[Learn more](#)

MEMBERSHIP AVAILABLE

Become an integral part of the PyTorch Foundation, to build and shape the future of AI.

[Join](#)

<https://pytorch.org/>

★ 189k TF 2.19 버전 출시됨

엔드 투 엔드 머신러닝 플랫폼

TensorFlow 설치



TensorFlow 시작하기

TensorFlow를 사용하면 어떤 환경에서도 실행할 수 있는 ML 모델을 쉽게 만들 수 있습니다. 대화형 코드 샘플을 통해 직관적인 API를 사용하는 방법을 알아보세요.

튜토리얼 보기 >

```
import tensorflow as tf
mnist = tf.keras.datasets.mnist

(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0

model = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(10, activation='softmax')
])
```

<https://www.tensorflow.org/?hl=ko>

Cloud 기반 ML 개발 환경

- 개인의 로컬 환경에 머신러닝 학습을 위한 세팅이 어려운 경우(GPU 성능, 로컬환경 버전 차이 등), 클라우드 환경을 이용하기도 함.
 - Amazon AWS (<https://aws.amazon.com/ko/>)
 - Microsoft Azure (<https://azure.microsoft.com/ko-kr/>)
 - 네이버 클라우드 (<https://www.ncloud.com/>)
 - Google Cloud (<https://cloud.google.com/?hl=ko>)
 - etc

Cloud 기반 ML 개발 환경

[Get Started](#)[Ecosystem](#) ▾[Mobile](#)[Blog](#)[Tutorials](#)[Docs](#) ▾[Resources](#) ▾[GitHub](#)

START VIA CLOUD PARTNERS

Cloud platforms provide powerful hardware and infrastructure for training and deploying deep learning models. Select a cloud platform below to get started with PyTorch.

	Alibaba Cloud	>
	Amazon Web Services	>
	Google Cloud Platform	>
	Microsoft Azure - <i>PyTorch Enterprise Program</i>	>

Cloud 기반 ML 개발 환경 (MS Azure)



Azure

탐색 ▾

제품 ▾

솔루션 ▾

가격 ▾

파트너 ▾

리소스 ▾

체험 계정

검색



Docs

지원

영업팀에 문의



모든 제품 검색

모든 제품 보기(200+개)

결과가 즉시 표시됨

추천

AI + 머신 러닝

분석기능

컴퓨팅

컨테이너

데이터베이스

DevOps

개발자 도구

하이브리드 + 다중 클라우드

ID

통합

IOT(사물 인터넷)

관리 및 거버넌스

추천

Azure 인기 제품 살펴보기



가상 머신

빠른 속도로 Windows 및 Linux 가상 머신 프로비전



Azure Virtual Desktop

어디서나 안전한 원격 데스크톱 환경 사용



Azure SQL

클라우드에서 항상 최신 상태로 유지되는 관리형 SQL 인스턴스



Azure Cosmos DB

모든 규모를 위한 개방형 API가 포함된 빠른 NoSQL 데이터베이스



AKS(Azure Kubernetes Service)

관리되는 Kubernetes를 사용하여 빌드 및 스케일링



Azure Cognitive Services

고품질 AI 모델을 API로 배포



App Service

강력한 웹 및 모바일용 클라우드 앱을 신속하게 구축



Azure PlayFab

단일 플랫폼에서 라이브 게임을 빌드 및 운영하는 데 필요한 모든 것



Azure Function

서버를 사용하지 않는 코드로 이벤트 처리



Azure Quantum

Accelerate your development lifecycle by building your quantum solution once and running it on multiple systems with little to no

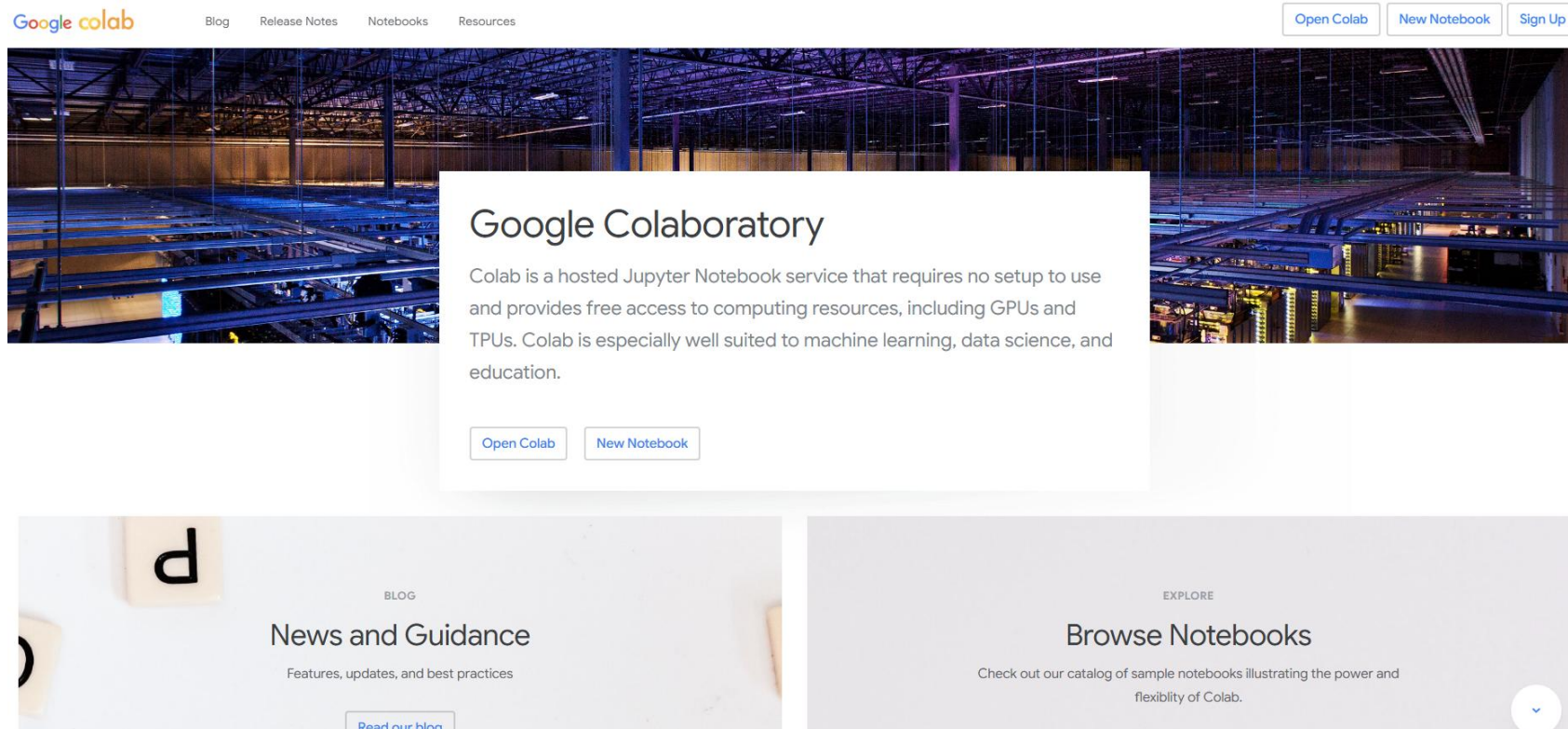


Azure Arc

어디서나 Azure 관리 및 서비스 확장

Google Colab

- 이번 프로젝트에서는 구글 코랩 사용
 - <https://colab.google/>



What is Google Colab?

- 구글 코랩은 구글 colaboratory 서비스의 줄임말
- 구글 코랩은 클라우드 기반으로 주피터 노트북 개발환경
- 코랩은 구글 드라이브, 도커, 리눅스, 구글 클라우드 등 기술로 이루어져 있음
- 웹 python을 작성하고 실행 가능
- 자신브라우저에서의 컴퓨터에 GPU가 없거나 저사양이라면 사용하는 것을 추천함
- 코랩 사용을 위한 팁
 - <https://ffoorreeuunn.tistory.com/244>

Google Colab

- 별도의 Python 설치가 필요 없음
- 데이터에 분석 사용되는 Tensor Flow, PyTorch, Keras, matplotlib, scikit-learn, pandas와 같은 패키지가 기본적으로 설치되어 있음
- **GPU를 무료로** 사용 가능
- Jupyter 노트북과 비슷하지만 더 좋은 기능을 제공함
- Git과 연동이 가능하여 사람들과 **협업**하여 코딩이 가능함

Anadonda + Jupyter Notebook

- Instead of Google Colab, you can use
 - Anaconda + Jupyter Notebook
 - Python과 ML Package인 PyTorch (Tensor Flow)를 편리하게 사용하게 하는 도구
 - 우리는 설치하지 않습니다 (너무 복잡한 관계로...)
 - 우리는 Google Colab을 사용합니다.
- <https://annajang.tistory.com/26>
- <https://hogni.tistory.com/29>
- <https://velog.io/@sekkaro96/Anaconda-Python%EA%B3%BC-Jupyter-notebook-%EC%84%A4%EC%B9%98>

Google Colab 시작하기

 Colaboratory에 오신 것을 환영합니다
파일 수정 보기 삽입 런타임 도구 도움말

☰ 목차 X

🔍 시작하기
📂 데이터 과학
🔗 머신러닝
{x} 추가 리소스
📁 추천 예시
+ 섹션

+ 코드 + 텍스트 Drive로 복사 연결 ▼

Colab 시작 페이지

Colab에 이미 익숙하다면 이 동영상을 통해 양방향 테이블, 코드 실행 기록 보기, 명령어 팔레트에 관해 알아보세요.



Colab이란?

Colaboratory(줄여서 'Colab'이라고 함)을 통해 브라우저 내에서 Python 스크립트를 작성하고 실행할 수 있습니다.

- 구성이 필요하지 않음
- GPU 무료 액세스
- 간편한 공유

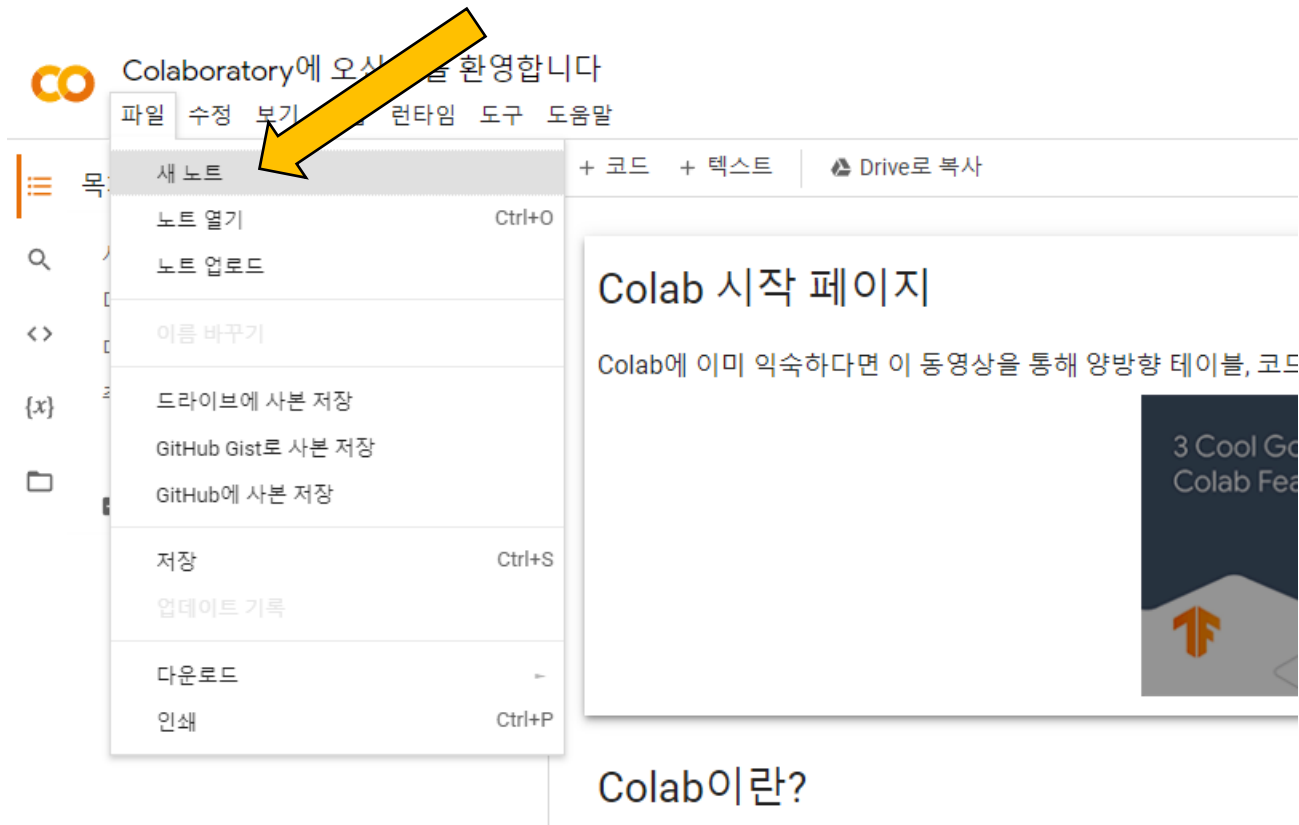
학생이든, 데이터 과학자든, AI 연구원이든 Colab으로 업무를 더욱 간편하게 처리할 수 있습니다. [Colab 소개 영상](#)에서 자세한 !

<https://colab.research.google.com/>

Google Colab 사용법

<https://colab.research.google.com/> 접속 후 로그인

파일 - 새 노트 클릭



Colab이란?

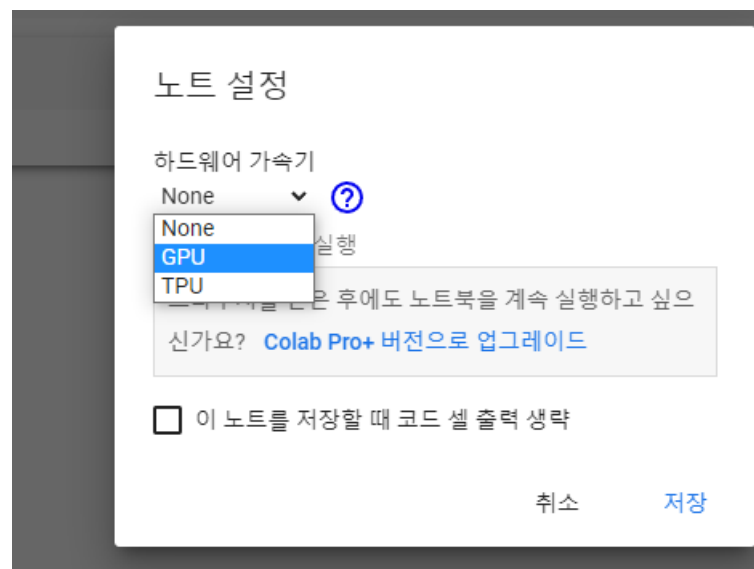
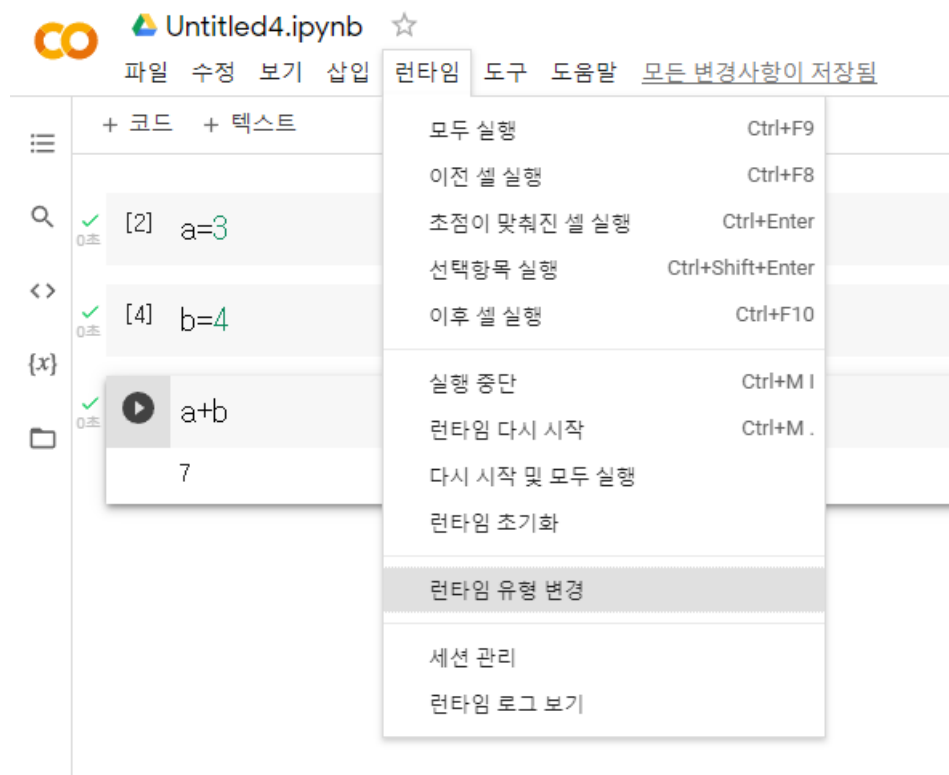
Google Colab 사용법

+코드를 누르고 작성 후 재생 버튼 누르면 한줄씩 실행 가능 (주피터 노트북 개발환경)



Google Colab 사용법

- GPU 사용
- 런타임 - 런타임 유형 변경 - GPU - 저장
- GPU와 TPU 사용은 하루 사용제한 있음



Project 3: Problems

- Problem 1. Create a maze environment using MiniGrid
- Problem 2. Solve maze problem with SARSA and Q-Learning
- Problem 3. Solve maze problem with DQN

Problem 1

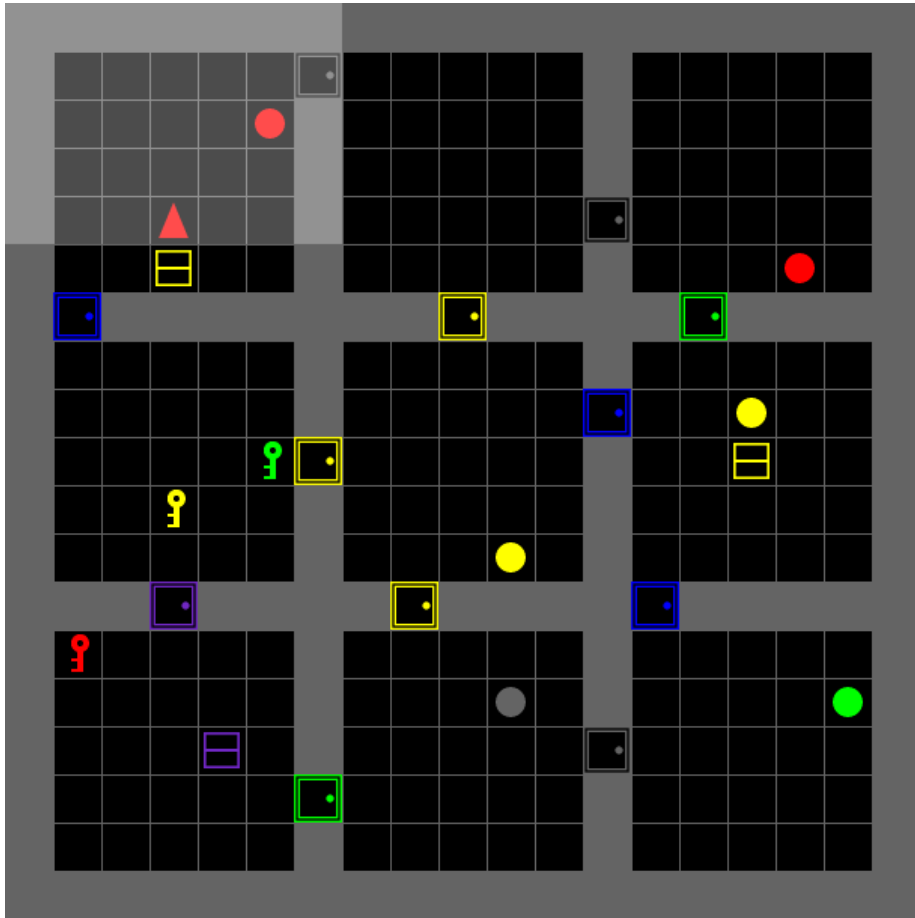
Problem 1. Create a maze environment using MiniGrid

- Problem 1. Design a maze environment based on the following Minigrid environment (see the next slide).



<https://minigrid.farama.org/>

Problem 1. What is Minigrid Environment?



Minigrid Environment

- Classic discrete action space RL environment.
- Benchmark for RL algorithms under discrete action conditions.

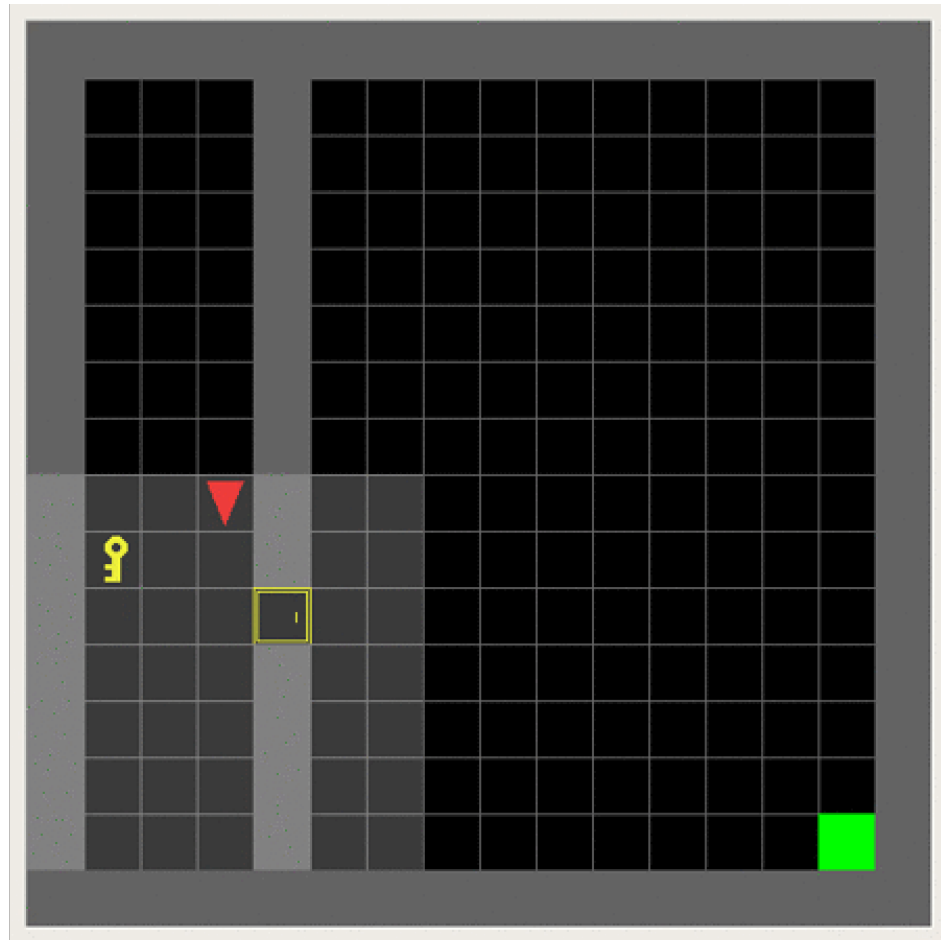
Agent Task

- Select actions from a discrete set
- Navigate the grid world
- Pick up a key
- Open a door
- Make a series of sequential decisions to reach the goal

Why MiniGrid?

- Simple, lightweight implementation
- Minimal code dependencies
- Easy to install

Problem 1. MiniGrid Environment



- Observation Space

Discrete(4) "Pick up the {color} {type}"
 $\{ \text{direction}, \text{image}, \text{mission} \}$
Box (0, 255, (7, 7, 3), uint8)

- Action Space

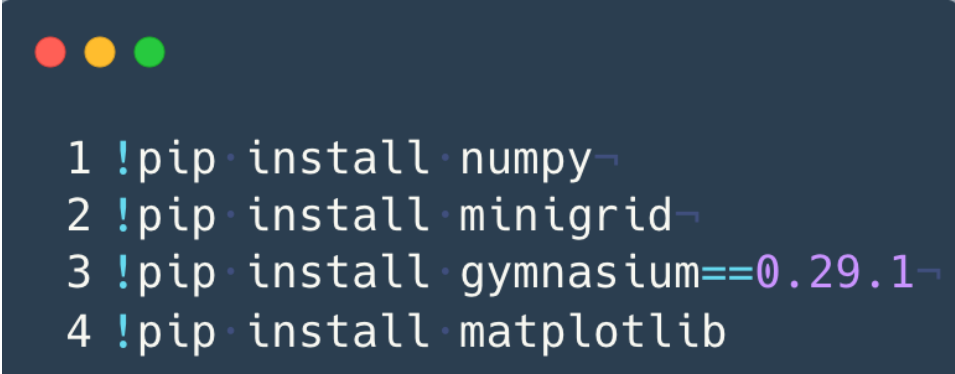
{Left, Right, Forward, Pickup, Drop, Toggle, Done}
Discrete (7)

- Objective:

Navigate the agent from the start position to the goal position in as few steps as possible.

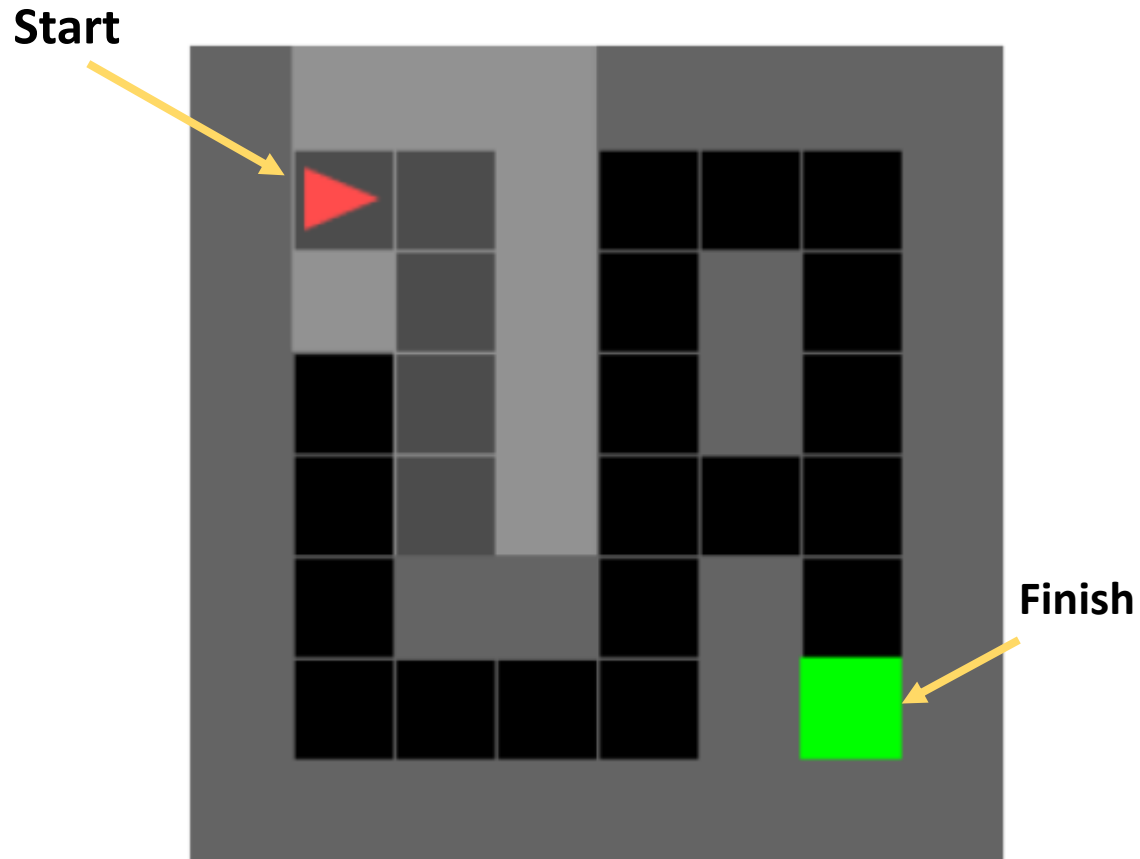
Problem 1: Required Python Libraries

- Numpy
- Minigrid
- Gymnasium
- Matplotlib



```
1 !pip install numpy↵
2 !pip install minigrid↵
3 !pip install gymnasium==0.29.1↵
4 !pip install matplotlib
```

Problem 1. Create a maze environment using MiniGrid



- 6×6 maze structure
- Initial state: (1, 1)
- Terminal state: (6, 6) (green)
- Walls: Gray-colored states
- Aisles: Black-colored states
- Three actions: Left, Right, Forward



Left



Right

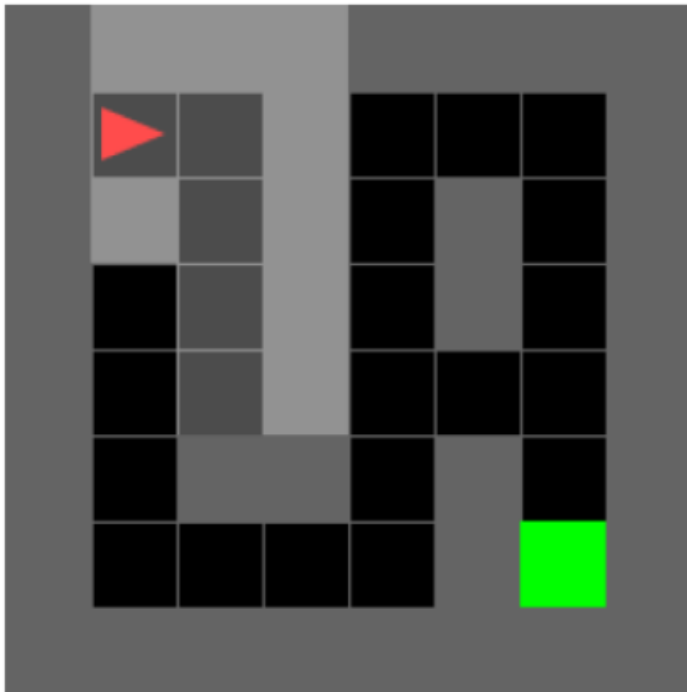


Forward

- Objective: Shortest path from start to finish

Problem 1. Create a maze environment using MiniGrid

- Verify the environment



14) DQN 학습 성능 저하
→ 위치 랜덤 샘플링
해결

```
env = SimpleEnv(render_mode="rgb_array")  
env.reset()  
image_data = env.render()  
image_data.shape  
plt.imshow(image_data)  
plt.axis("off")
```

Problem 2

Problem 2. Solve the maze problem with SARSA and Q-Learning

- Problem 2-1. Solve the maze problem using Q-Learning and then provide the state trajectory(=path) result generated by Q-Learning. Is it optimal?
- Problem 2-2. Solve the maze problem using SARSA and then provide the state trajectory(=path) result generated by SARSA. Is it optimal?

Problem 2. Solve the maze problem with SARSA and Q-Learning

- Install gymnasium minigrid environment

```
!pip install gymnasium==0.29.1 minigrid
```

- Drive mount to save the parameter

```
from google.colab import drive  
drive.mount('/content/drive')  
%cd drive/My Drive/Project
```

Problem 2: Required Python Libraries

- Math
- Gym
- Numpy
- Pytorch
- Minigrid
- Encoder
- Tqdm
- Matplotlib

```
from __future__ import annotations

import io
import time
import imageio
import math
from abc import *
import numpy as np
from tqdm import tqdm
from base64 import b64encode
import matplotlib.pyplot as plt
from IPython.display import HTML, display, clear_output
import gymnasium as gym
from minigrid.core.grid import Grid
from minigrid.core.constants import COLOR_NAMES
from minigrid.manual_control import ManualControl
from minigrid.minigrid_env import MiniGridEnv, MissionSpace
from minigrid.core.world_object import Door, Goal, Key, Wall
from matplotlib.animation import FuncAnimation
```

Problem 2-1. Solve the maze problem with Q-Learning

- Minigrid Environment

```
class SimpleEnv(MiniGridEnv):
    def __init__(self, size=8, agent_start_pos=(1, 1), agent_start_dir=0, max_steps: int | None = None, **kwargs):
        self.agent_start_pos = agent_start_pos
        self.agent_start_dir = agent_start_dir

        mission_space = MissionSpace(mission_func=self._gen_mission)

        super().__init__(mission_space=mission_space, grid_size=size, max_steps=100, **kwargs)

    @staticmethod
    def _gen_mission():
        return "grand mission"

    def _gen_grid(self, width, height):
        self.grid = Grid(width, height)

        self.grid.wall_rect(0, 0, width, height)

        self.mission = "grand mission"
```

Set wall and start position

Problem 2-1. Solve the maze problem with Q-Learning

- Epsilon greedy

- Epsilon decay

$$\begin{cases} \text{Random action } a_t & \text{with probability } \epsilon \\ a_t = \underset{a}{\operatorname{argmax}} Q(s_t, a_t) & \text{with probability } 1 - \epsilon \end{cases}$$

- Reward plot

```
def eps_greedy(Q_values, state, env, epsilon=1.0, epsilon_decay=0.995):  
      
    return action, epsilon
```

```
def reward_plot(total_reward):  
  
    plt.plot(total_reward, label="Total Reward per Episode")  
    plt.xlabel("Episodes")  
    plt.ylabel("Total Reward")  
    plt.legend()
```

Problem 2-1. Solve the maze problem with Q-Learning

- Action decision
- Update the state
- Reward
- Update Q-values with TD error
- Save the Q-values

```
def run(env, goal, Q_values, n_episodes, epsilon, epsilon_decay, gamma, total_reward, lr):  
    for i in tqdm(range(n_episodes), desc="Simulating environment", unit="step"):  
        obs, info = env.reset(seed=42)  
        state = np.concatenate((env.agent_pos[0]-1, env.agent_pos[1]-1, obs["direction"]), axis=None)  
        done = False  
        reward_sum = 0  
  
        while not done:  
            Action decision  
            next_obs, reward, terminated, truncated, info = env.step(action)  
  
            next_state = np.concatenate((env.agent_pos[0]-1, env.agent_pos[1]-1, next_obs["direction"]), axis=None)  
  
            Reward  
  
            Update Q-values with TD error  
  
            state = next_state  
            reward_sum += reward  
            done = terminated or truncated  
            # terminated: 사용자가 원하는 종료조건에 의해 종료된 경우,  
            # truncated: 환경이 최대 step수에 도달하여 종료된 경우  
            if done:  
                total_reward.append(reward_sum)  
  
        if i % 10000 == 0:  
            np.save('/content/drive/MyDrive/Project/Q_values_save', Q_values)  
        env.close()  
    return Q_values, total_reward
```

Problem 2-1. Solve the maze problem with Q-Learning

- Parameter setup

```
def main():
    #define training parameters
    env = SimpleEnv(render_mode="rgb_array") # human, rgb_array, ansi
    Q_values_init = np.zeros((env.width-2, env.height-2, 4, 3))
    goal = [env.width-3, env.height-3]

    epsilon = 1.0
    epsilon_decay = 
    gamma = 0.99 #discount factor
    lr = 0.0004 #learning rate
    n_episodes = 
    total_reward = []

    Q_values, total_reward = run(env, goal, Q_values_init, n_episodes, epsilon, epsilon_decay, gamma, total_reward, lr)
    reward_plot(total_reward)
    np.save('/content/drive/MyDrive/Project/Q_values_save', Q_values)
    print('Training complete!')

if __name__ == "__main__":
    main()
```

Problem 2-1. Solve the maze problem with Q-Learning

- Display the result video

```
# Formally, In colab is not working "human" rendering option
# Therefore, we should visualize through below function.
def display_video(frames, fps=10):
    video_buffer = io.BytesIO()

    imageio.mimsave(video_buffer, frames, format="mp4", fps=fps)
    video_buffer.seek(0)

    video_base64 = b64encode(video_buffer.read()).decode()
    video_html = f"""
    <video width="640" height="480" controls>
        <source src="data:video/mp4;base64,{video_base64}" type="video/mp4">
    </video>
    """
    return HTML(video_html)
```

Problem 2-1. Solve the maze problem with Q-Learning

- Test the learned Q values

```
env = SimpleEnv(render_mode="rgb_array") # human, rgb_array, ansi
obs, info = env.reset(seed=42)
state = np.concatenate((env.agent_pos[0]-1, env.agent_pos[1]-1, obs["direction"]), axis=None)
done = False
frames = []
frames.append(env.render())
i = 0
Q_values_load = np.load('/content/drive/MyDrive/Project/Q_values_save.npy')

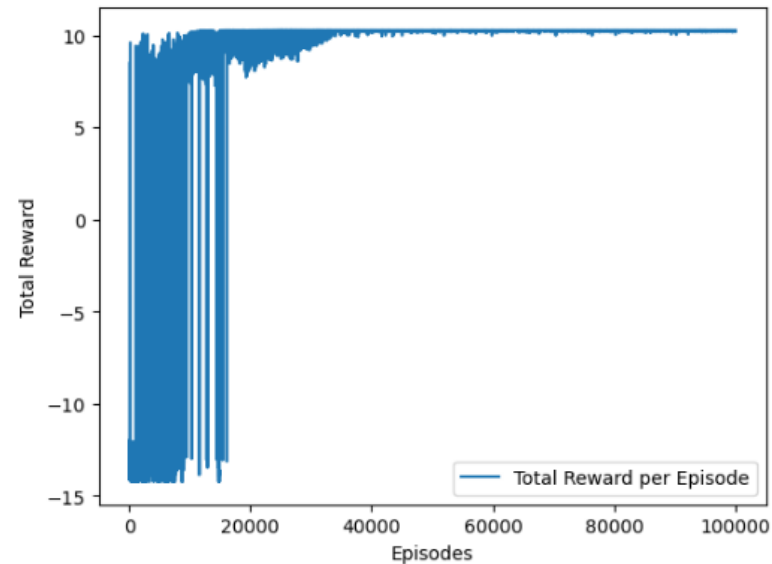
while not done:
    Action decision, epsilon = 0
    next_obs, reward, terminated, truncated, info = env.step(action)
    state = np.concatenate((env.agent_pos[0]-1, env.agent_pos[1]-1, next_obs["direction"]), axis=None)
    done = terminated or truncated
    frames.append(env.render())
    time.sleep(0.03)
    i += 1

env.close()
print(f" Finished at the {i}-th iteration")
display_video(frames) # runtime 옵션을 TPU로 하면 에러남. GPU로 돌려야함.
```

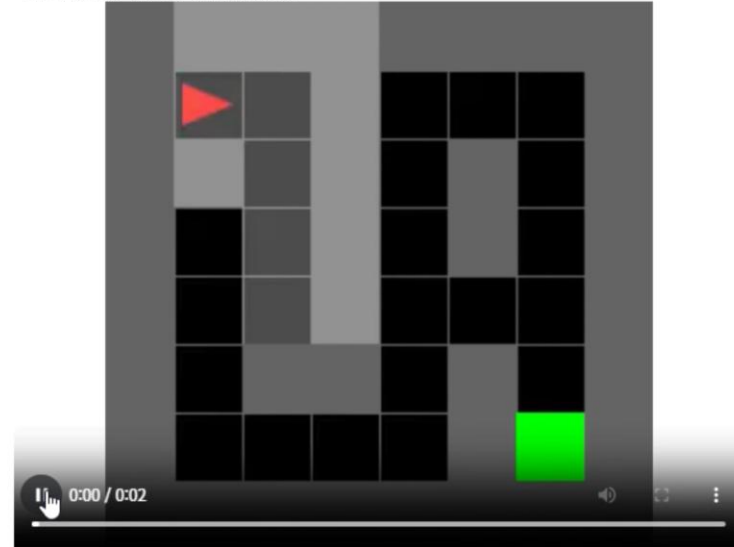
Problem 2-1. Solve the maze problem with Q-Learning

- Reward plot and Result video

Simulating environment: 100% ██████████ 100000/100000 [13:49<00:00, 120.60step/s]
Training complete!



Finished at the 23-th iteration



Problem 2-2. Solve the maze problem with SARSA

- Except for the TD error computation and Q-value update, it is almost identical to Q-learning.

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\text{terminal-state}, \cdot) = 0$

Repeat (for each episode):

Initialize S

Repeat (for each step of episode):

Choose A from S using policy derived from Q (e.g., ϵ -greedy)

Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

until S is terminal

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\text{terminal-state}, \cdot) = 0$

Repeat (for each episode):

Initialize S

Choose A from S using policy derived from Q (e.g., ϵ -greedy)

Repeat (for each step of episode):

Take action A , observe R, S'

Choose A' from S' using policy derived from Q (e.g., ϵ -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A'$

until S is terminal

Problem 2-2. Solve the maze problem with SARSA

- Action decision
- Update the state
- Reward
- Update Q-values with TD error
- Save the Q-values

```
def run(env, goal, Q_values, n_episodes, epsilon, epsilon_decay, gamma, total_reward, lr):  
    for i in tqdm(range(n_episodes), desc="Simulating environment", unit="step"):  
        obs, info = env.reset(seed=42)  
        state = np.concatenate((env.agent_pos[0]-1, env.agent_pos[1]-1, obs["direction"]), axis=None)  
        done = False  
        reward_sum = 0  
  
        while not done:  
  
            action = next_action  
            state = next_state  
            reward_sum += reward  
            done = terminated or truncated  
            # terminated: 사용자가 원하는 종료조건에 의해 종료된 경우,  
            # truncated: 환경이 최대 step수에 도달하여 종료된 경우  
            if done:  
                total_reward.append(reward_sum)  
  
        if i % 10000 == 0:  
            np.save('/content/drive/MyDrive/Project/SARSA_save', Q_values)  
            env.close()  
    return Q_values, total_reward
```

Problem 3

Problem 3. Solve the maze problem with DQN

- Problem 3. Solve the maze problem using DQN algorithm. Explain how to implement DQN (neural network, replay buffer and overall DQN algorithm).

Problem 3: Required Python Libraries

- Minigrid wrapper
- Pytorch

```
from __future__ import annotations

import io
import time
import imageio
import math, random
from abc import *
import numpy as np
from tqdm import tqdm
from base64 import b64encode
import matplotlib.pyplot as plt
from IPython.display import HTML, display, clear_output
from minigrid.core.grid import Grid
from minigrid.core.constants import COLOR_NAMES
from minigrid.manual_control import ManualControl
from minigrid.minigrid_env import MiniGridEnv, MissionSpace
from minigrid.core.world_object import Door, Goal, Key, Wall
from minigrid.wrappers import RGBImgObsWrapper
from matplotlib.animation import FuncAnimation

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from collections import deque
import torchvision.transforms as transforms
```

Guidelines: Problem 3. DQN Implementation

1) To implement DQN, first define **Replay Buffer** as a **Class** in Python, which contains two main functions, (i) **Push** and (ii) **Sample**.

- **Push** function stores a **state** transition (state, action, reward, next state, done) in the replay buffer
 - Input: state transition, Output: None
- **Sample** function obtains M state transition samples from the replay buffer.
 - Input: Number of samples M , Output: M state transition samples

```
class ReplayBuffer(object):
    def __init__(self, capacity):
        self.buffer = deque(maxlen=capacity)

    def push(self, state, action, reward, next_state, done):
        self.buffer.append((state, action, reward, next_state, done))

    def sample(self, batch_size):
        state, action, reward, next_state, done = zip(*random.sample(self.buffer, batch_size))
        return torch.stack(state), action, reward, torch.stack(next_state), done

    def __len__(self):
        return len(self.buffer)
```

Guidelines: Problem 3. DQN Implementation

2) Epsilon Decay

- Since DQN is an off-policy algorithm, DQN agent can learn from experiences generated under different policies.
- Hence, it is recommended to use epsilon, which decreases as the number of episodes increases to improve exploitation of DQN.

```
epsilon = 1.0
```

```
epsilon_decay = 
```

Epsilon initialization

```
epsilon = max(epsilon*epsilon_decay, 0.01) # 0.01은 epsilon_min
```

[ϵ -greedy policy]

$\begin{cases} \text{Random action } a_t & \text{with probability } \epsilon \\ a_t = \underset{a}{\operatorname{argmax}} Q(s_t, a) & \text{with probability } 1 - \epsilon \end{cases}$

Guidelines: Problem 3. Neural Networks and Loss Functions

- 3) To implement neural networks for DQN, the 'Pytorch' library is required.
- 4) Since DQN obtains Q-values using neural networks, computing the TD error also requires the 'Pytorch' library.
- 5) To understand how to define neural networks and how to calculate TD error using Pytorch, please refer to the following URL.

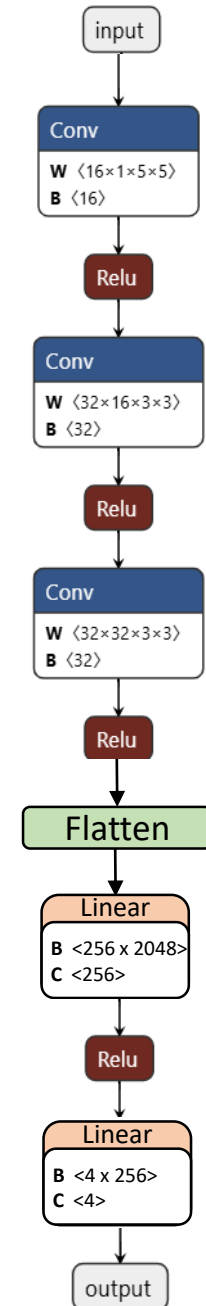
URL: https://pytorch.org/tutorials/beginner/blitz/neural_networks_tutorial.html

Guidelines: Problem 3. Neural Networks and Loss Functions

- Neural Networks: Convolutional Neural Network for image processing

```
class Model(nn.Module):  
    def __init__(self, num_inputs, num_actions):  
        super(Model, self).__init__()  
  
        self.conv1 =   
        self.conv2 =   
        self.conv3 =   
  
        # Fully connected layers  
        self.fc1 =   
        self.fc2 =   
  
    def forward(self, x):  
        x = F.relu(self.conv1(x))  
        x = F.relu(self.conv2(x))  
        x = F.relu(self.conv3(x))  
        x = x.view(x.size(0), -1) # Flatten  
        x = F.relu(self.fc1(x))  
        return self.fc2(x)
```

CNN



Guidelines: Problem 3. Neural Networks and Loss Functions

- Preprocessing : Image preprocessing is necessary to improve the success rate of image training
 - Scaling
 - Grayscale
 - Etc...

```
def pre_state(obs):  
    state = obs['image'] / 255.0 # RGB 최대값이 255이므로 scaling 해준 것, (64,64,3)  
    state = torch.tensor(state, dtype=torch.float32).permute(2, 0, 1)  
    to_grayscale = transforms.Grayscale(num_output_channels=1) # rgb 3개 채널은 데이터 처리에 시간이 오래걸리므로 흑백사진(채널수=1)으로 변환  
    state = to_grayscale(state) # (1,64,64)  
    return state
```

Guidelines: Problem 3. Neural Networks and Loss Functions

- TD Errors

```
def compute_td_error(batch_size, replay_buffer, model, target_model, gamma, optimizer):
    if batch_size > len(replay_buffer):
        # Replay Buffer 크기가 batch_size보다 작은 상태에서는 미니배치를 샘플링할 수 없음.
        # 따라서 Replay Buffer가 일정 개수 이상 채워질 때까지 학습을 하지 않음.
        return None

    state, action, reward, next_state, done = replay_buffer.sample(batch_size)
    state = torch.FloatTensor(state).to(device)
    reward = torch.FloatTensor(reward).unsqueeze(1).to(device)
    action = torch.LongTensor(action).unsqueeze(1).to(device)
    next_state = torch.FloatTensor(next_state).to(device)
    mask = torch.FloatTensor(1 - np.float32(done)).unsqueeze(1).to(device)

    q_values = model(state)
    q_value = q_values.gather(1, action)

    next_q_values = target_model(next_state)
    next_q_value = next_q_values.max(1)[0].detach().unsqueeze(1)

TD error update  

        Compute loss function



    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    return loss
```

Guidelines: Problems 3. DQN algorithm

6) Determine hyperparameters

(minibatch size, replay buffer size, number of episodes, discount factor, target network update period, ...)

```
#define training parameters
env = SimpleEnv(render_mode="rgb_array") # human, rgb_array, ansi
env = RGBImgObsWrapper(env)

model = Model((1, 64, 64), 3).to(device) # policy network, Model(state_size, action_size)
target_model = Model((1, 64, 64), 3).to(device) # target network
update_target(model, target_model) # Initialize networks
target_model.eval()

epsilon = 1.0
epsilon_decay = 
gamma = 0.99 #discount factor
lr = 1e-4 #learning rate
n_episodes = 

batch_size = 
Target_Update =  # target network update period
optimizer = optim.RMSprop(model.parameters(), lr=lr)
replay_buffer = ReplayBuffer(10000)
```

Guidelines: Problems 3. DQN algorithm

8) Select action with epsilon greedy method

```
def eps_greedy(model, state, epsilon):  
    if np.random.random() < epsilon:  
          
    else:  
        state = torch.FloatTensor(state).unsqueeze(0).to(device)  
          
    return action
```

[ϵ -greedy policy]

$\begin{cases} \text{Random action } a_t & \text{with probability } \epsilon \\ a_t = \underset{a}{\operatorname{argmax}} Q(s_t, a_t) & \text{with probability } 1 - \epsilon \end{cases}$

9) Update target network

For the stability of the training model, it is important to update the target network at a reasonable frequency.

```
def update_target(current_model, target_model):  
    target_model.load_state_dict(current_model.state_dict())
```

Guidelines: Problems 2-1. DQN algorithm

10) Follow the DQN algorithm

Randomizing the agent's initial position in every episode can improve training performance

Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function Q^- with weights $\theta^- = \theta$

For episode = 1, M **do**

Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, T$ **do**

With probability ϵ select a random action a_t
otherwise select $a_t = \operatorname{argmax}_a Q(\phi(s_t), a; \theta)$

Execute action a_t in emulator and observe reward r_t and image x_{t+1}

Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ

Every C steps reset $\hat{Q} = Q$

End For

End For

```
def run(env, model, target_model, optimizer, replay_buffer, Target_Update, batch_size, n_episodes, epsilon, epsilon_decay, gamma):
    goal = [env.width-3, env.height-3]
    total_reward = []
    bar = tqdm(range(n_episodes))
```

```
for i in bar:
    obs, info = env.reset()
```

```
state = pre_state(env.observation(obs))
distance = math.sqrt((goal[0] - env.agent_pos[0]-1) ** 2 + (goal[1] - env.agent_pos[1]-1) ** 2)
done = False
reward_sum = 0
```

```
while not done:
```

Action decision

```
next_obs, reward, terminated, truncated, info = env.step(action)
next_state = pre_state(next_obs)
next_distance = math.sqrt((goal[0] - env.agent_pos[0]-1) ** 2 + (goal[1] - env.agent_pos[1]-1) ** 2)
```

Additional reward

Store transition in the Replay buffer

Update TD error and compute loss

```
distance = next_distance
state = next_state
```

```
reward_sum += reward
done = terminated or truncated
# terminated: 사용자가 원하는 종료조건에 의해 종료된 경우,
# truncated: 환경이 최대 step수에 도달하여 종료된 경우
```

```
if done:
    total_reward.append(reward_sum)
```

Epsilon decay

```
if (i % Target_Update == 0) and (i != 0): # Target_Update 주기마다 target network 업데이트
    update_target(model, target_model)
```

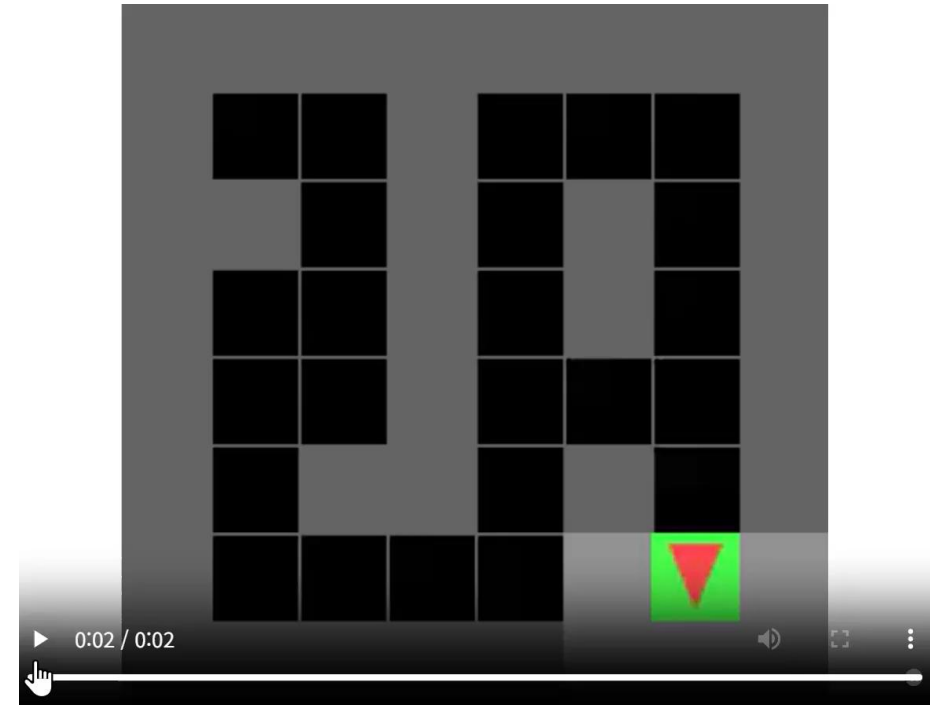
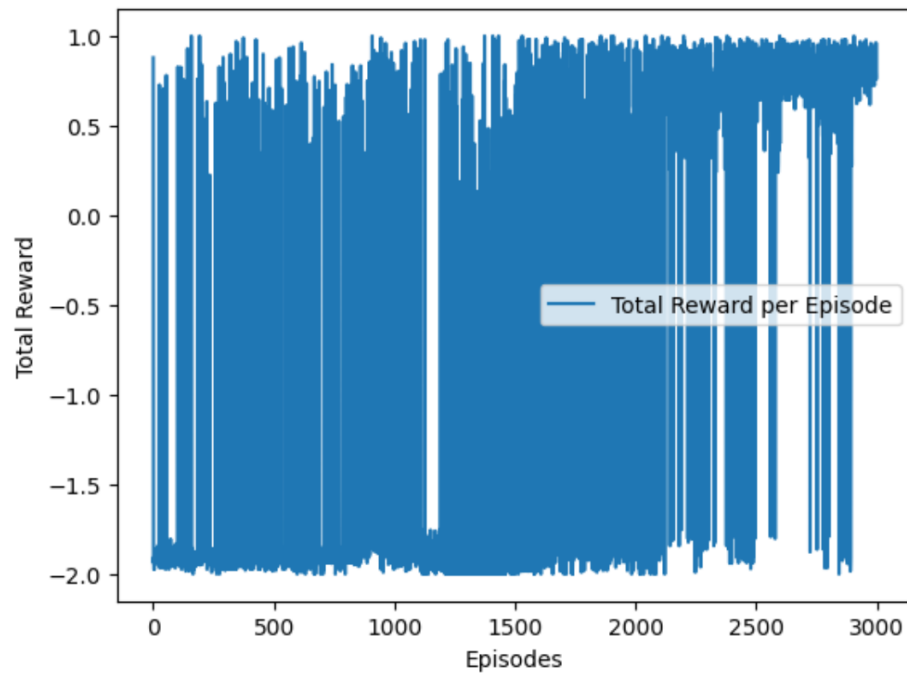
```
if i % 500 == 0:
    torch.save(model.state_dict(), './content/drive/MyDrive/Project/DQN_save.pth')
```

```
bar.set_description(f'Simulating environment: {i+1}, Episode reward: {reward_sum}, epsilon: {epsilon}')
```

```
env.close()
return model, total_reward
```

Guidelines: Problems 2-1. DQN algorithm

11) Obtain the results generated by DQN after training.



Guidelines: Problems 3. Test

1) Test the trained model

```
env = SimpleEnv(render_mode="rgb_array") # human, rgb_array, ansi
env = RGBImgObsWrapper(env)
obs, info = env.reset()
state = pre_state(obs)
done = False
frames = []
frames.append(env.render())
i = 0
model_load = Model((1, 64, 64), 3).to(device)
model_load.load_state_dict(torch.load('/content/drive/MyDrive/Project/DQN_save.pth'))
model_load.eval()

while not done:
    Action decision, epsilon = 0

    next_obs, reward, terminated, truncated, info = env.step(action)
    state = pre_state(next_obs)
    done = terminated or truncated
    frames.append(env.render())
    time.sleep(0.03)
    i += 1

print(terminated)
print(env.agent_pos)
env.close()
print(f"Finished at the {i}-th iteration")
display_video(frames) # runtime 옵션을 TPU로 하면 에러남. GPU로 돌려야함.
```